

Machine Learning for Quant Research

Supervised and Unsupervised Learning, Neural Networks, Feature Engineering, and Model Validation for Markets

Statistical Learning Applied to Prediction and Pattern Detection — and Why Backtesting Discipline Must Come First

A Level 3 module in the Quantitative Finance Curriculum

Covers: supervised learning (linear/logistic regression, trees, random forests, gradient boosting/XGBoost, SVM) · unsupervised learning (PCA, clustering, regime identification) · neural networks (feed-forward, CNN/RNN/LSTM/Transformer — awareness) · feature engineering/label construction/prediction horizons/probabilistic forecasts/validation/feature importance/SHAP/model drift/retraining

Table of Contents

How to Use This Guide

What You Need to Know Before Starting

Understand Before Proceeding

Why It Matters

How This Connects to the Rest of the Curriculum

Glossary of Recurring Terms

Part 1 — Supervised Learning for Prediction

1.1 Linear and Logistic Regression

1.2 Trees, Random Forests, and Gradient Boosting (XGBoost)

1.3 Support Vector Machines

Part 2 — Unsupervised Learning for Structure Discovery

2.1 PCA and Dimensionality Reduction

2.2 Clustering

2.3 Regime Identification

Part 3 — Neural Networks (Awareness)

3.1 Feed-Forward Networks

3.2 CNN, RNN/LSTM, and Transformer Architectures (Awareness)

Part 4 — The ML Research Pipeline: Features, Labels, and Horizons

4.1 Feature Engineering

4.2 Label Construction and Prediction Horizons

4.3 Probabilistic Forecasts

Part 5 — Validating and Monitoring ML Models

5.1 Validation for ML Signals

5.2 Feature Importance and SHAP

5.3 Model Drift and Retraining

Practical Exercise

Common Beginner Mistakes

Advanced Extensions

Professional Applications

Where to Go Next

How to Use This Guide

This document is a companion to **Machine Learning for Quant Research**, the Level 3 module covering statistical learning methods applied to prediction and pattern-detection problems in markets. It surveys the standard supervised and unsupervised methods, gives an awareness-level tour of neural network architectures, and then spends real weight on the parts of the pipeline — feature engineering, label construction, validation, and monitoring — that determine whether an ML model produces a genuine trading edge or an expensive, well-disguised way to overfit noise.

The ordering of this module is deliberate: methods come first so you have the vocabulary, but the pipeline and validation material in Parts 4 and 5 is where the real risk lives, and is written to be read carefully rather than skimmed. Do not skip **Understand Before Proceeding** below.

What You Need to Know Before Starting

- **Regression** — ordinary least squares, coefficients, and residuals; almost every method in Part 1 is best understood as a generalization or extension of this basic idea.
- **Backtesting** — especially overfitting, cross-validation, and multiple testing; this module assumes you have already internalized why a biased or leaky validation procedure invalidates a result, and builds directly on the purged walk-forward methodology introduced there.
- **Statistics** — hypothesis testing and multiple-testing correction, since ML research typically involves searching over many more candidate features and model configurations than traditional strategy research, multiplying the same risk.

Understand Before Proceeding

***Prerequisite gate** — Do not build ML trading strategies before statistics and backtesting methodology are Depth 3+. And separately: do not reach for deep learning merely because it is sophisticated — try a simpler, interpretable model first, and only escalate if it demonstrably falls short. Both warnings point at the same underlying risk from different directions: ML models, and neural networks especially, have enormous capacity to fit patterns in data, including patterns that are pure noise. Without airtight validation discipline already in place, that capacity is a liability, not an advantage — it will find and confidently report a 'signal' in a dataset that has none.*

Why It Matters

ML can add real value for feature combination and non-linear pattern detection, but it dramatically amplifies overfitting risk if backtesting discipline is not already solid. This is not a hedge or a disclaimer — it is the central fact that should shape how much of this module's toolkit you actually reach for on a given problem. A linear regression with five hand-chosen, economically motivated features has limited capacity to overfit; a gradient-boosted ensemble with two hundred engineered features and hundreds of trees has enormous capacity to overfit, and will happily use that capacity if the validation procedure does not stop it.

The practical implication is a specific discipline, not a general caution: every technique in this module earns its place only if it demonstrably improves out-of-sample, cost-aware performance over a simpler baseline, evaluated with exactly the bias controls from the Backtesting module. A more sophisticated model that cannot clear that bar has not added value — it has added complexity, fragility, and a harder-to-diagnose failure mode when it eventually stops working.

How This Connects to the Rest of the Curriculum

- **Research** — ML is one more tool in the alpha-research toolkit, subject to exactly the same validation discipline as any other signal from the Alpha Research module — an ML signal does not get a pass on economic rationale or out-of-sample testing just because it is more sophisticated.
- **Development** — Feature stores, model registries, and retraining pipelines are Level 4 development responsibilities that ML research depends on — a research process that cannot reliably reproduce which features and which model version produced a given historical prediction is not a reproducible research process.
- **Trading** — Model drift monitoring is essential once an ML-based signal is live — markets are non-stationary and models decay, and an ML model's decay can be harder to detect early than a simpler rule-based strategy's, precisely because its behavior is less interpretable.

Glossary of Recurring Terms

Supervised learning — A class of methods that learn a mapping from input features to a known target (label), such as a future return or direction, using historical examples where both are observed.

Unsupervised learning — A class of methods that find structure in data — groupings, lower-dimensional representations, regimes — without a labelled target to predict.

Gradient boosting — An ensemble method (e.g., XGBoost) that builds a sequence of typically shallow trees, each trained to correct the errors of the ensemble built so far.

Random forest — An ensemble of decision trees, each trained on a bootstrapped sample of the data and a random subset of features, with predictions averaged (or voted) across trees.

Support vector machine (SVM) — A classifier that finds the decision boundary maximizing the margin between classes, optionally using a kernel to separate classes that are not linearly separable in the original feature space.

Principal component analysis (PCA) — A dimensionality-reduction technique that re-expresses correlated features as a smaller number of uncorrelated components, ordered by how much variance each explains.

Clustering — An unsupervised method that groups observations (e.g., assets or time periods) into clusters based on similarity in feature space, without predefined group labels.

Regime identification — The use of statistical or ML methods to detect distinct market states (e.g., high- vs. low-volatility regimes) from data, often as an input to conditioning other strategies.

Feed-forward neural network — A neural network in which information flows in one direction, through one or more hidden layers of weighted, nonlinearly-transformed combinations of the input, to an output.

Feature engineering — The process of transforming raw data into the input variables (features) actually presented to a model, including rolling statistics, ratios, lags, and other domain-informed transforms.

Label construction — The process of defining the target variable a supervised model is trained to predict, including its horizon and how ambiguous or borderline outcomes are handled.

Prediction horizon — The length of time between when a prediction is made and when its target outcome is realized and can be evaluated.

Probabilistic forecast — A prediction expressed as a probability or distribution over outcomes, rather than a single point estimate, allowing downstream position sizing to account for model confidence.

Calibration — The property that a model's predicted probabilities match observed frequencies — a model that predicts a 70% chance of an outcome should be right about that outcome roughly 70% of the time.

Feature importance — A measure of how much a given feature contributes to a model's predictions, used both for interpretation and for pruning uninformative features.

SHAP (SHapley Additive exPlanations) — A method for attributing an individual prediction to the contribution of each input feature, grounded in cooperative game theory, allowing per-prediction as well as aggregate interpretation.

Model drift — The degradation of a model's predictive performance over time as the statistical relationship between features and target changes in ways the model was not trained on.

Retraining — The process of refitting a model on more recent data, on a schedule or triggered by detected drift, to keep it aligned with the current market environment.

Part 1 — Supervised Learning for Prediction

Supervised learning methods learn a mapping from features to a known target using historical examples. This part covers the standard toolkit, roughly in order of increasing flexibility (and increasing overfitting risk) — starting with the interpretable baseline every more complex model should be measured against.

1.1 Linear and Logistic Regression

Linear regression predicts a continuous target — a forward return, say — as a weighted sum of input features. **Logistic regression** predicts the probability of a binary or categorical outcome — up vs. down — by passing that same weighted sum through a sigmoid function that maps it into the $[0, 1]$ range.

Figure 1.1 — Linear vs. Logistic Regression: Continuous vs. Probabilistic Targets

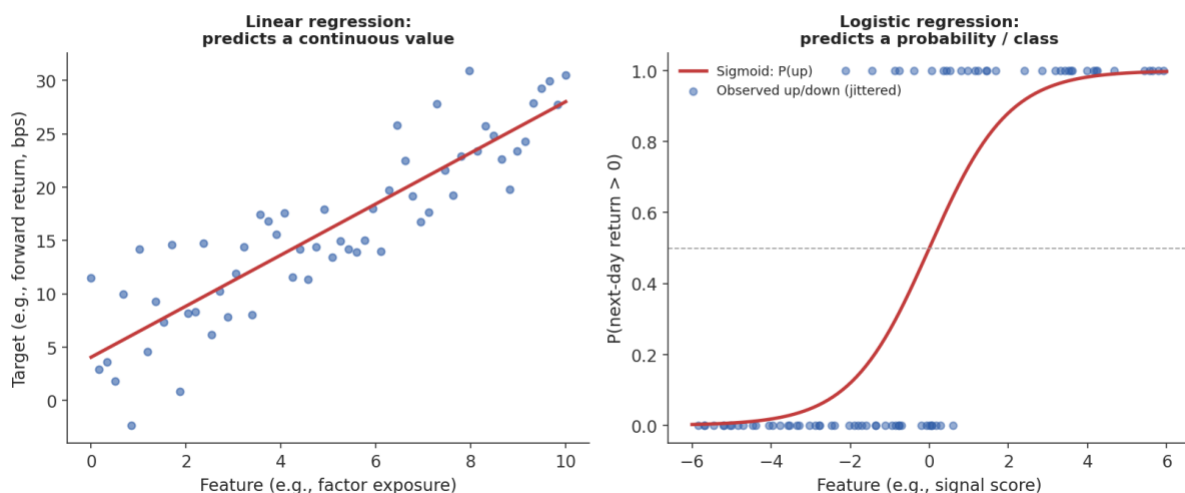


Figure 1.1 — Linear regression fits a continuous target directly; logistic regression maps a weighted feature sum through a sigmoid to produce a probability, suitable for a directional (up/down) target.

$$\text{Linear: } \hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n \quad \text{Logistic: } P(y=1) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)})$$

Both are the correct starting point for almost any new prediction problem in this module, not merely a beginner's simplification: their coefficients are directly interpretable (a unit increase in feature x changes the prediction by β units, holding other features fixed), they have limited capacity to overfit relative to more flexible models, and they train and validate quickly, which makes them an efficient way to establish a performance baseline before reaching for anything more complex.

1.2 Trees, Random Forests, and Gradient Boosting (XGBoost)

A **decision tree** predicts a target by asking a sequence of yes/no questions about the features, splitting the data at each step to make the resulting groups as pure (homogeneous in outcome) as possible, until reaching a leaf that outputs a prediction. A single tree is easy to visualize and interpret, but tends to overfit badly on its own.

Figure 1.2 — From a Single Tree to Gradient Boosting

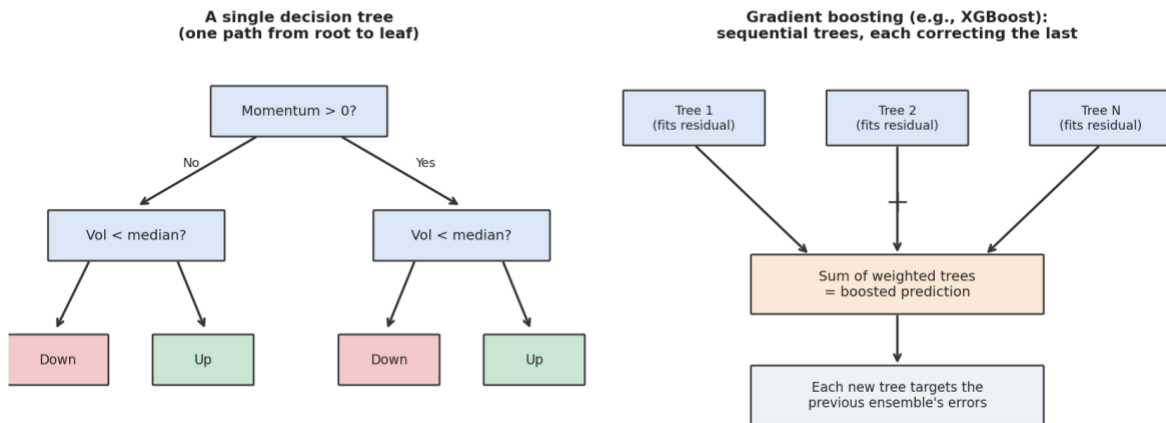


Figure 1.2 — A single decision tree makes one sequence of splits; gradient boosting builds many shallow trees in sequence, each one specifically trained to correct the errors of the trees built so far.

A **random forest** addresses a single tree's overfitting tendency by averaging many trees, each trained on a bootstrapped sample of the data and a random subset of features at each split — the randomness decorrelates the trees' individual errors, so the average is more stable than any one tree. **Gradient boosting** (implemented efficiently by libraries like XGBoost and LightGBM) takes a different approach to combining trees: rather than averaging many independent trees, it builds them sequentially, with each new tree specifically trained to predict the residual error of the ensemble built so far.

Tip — Gradient-boosted trees are, in practice, the most commonly used non-linear model family in quantitative finance for structured, tabular feature sets — they handle non-linear relationships and feature interactions well, are relatively robust to unscaled or mixed-type features, and (with SHAP, Section 5.2) remain interpretable at the level of individual predictions, which linear models are not always flexible enough to capture and pure neural networks are not always able to explain as cleanly.

1.3 Support Vector Machines

A **support vector machine (SVM)** finds the decision boundary that separates two classes with the maximum possible margin — the widest possible gap between the boundary and the nearest points of each class, called the **support vectors**, since they are the observations that actually determine where the boundary sits.

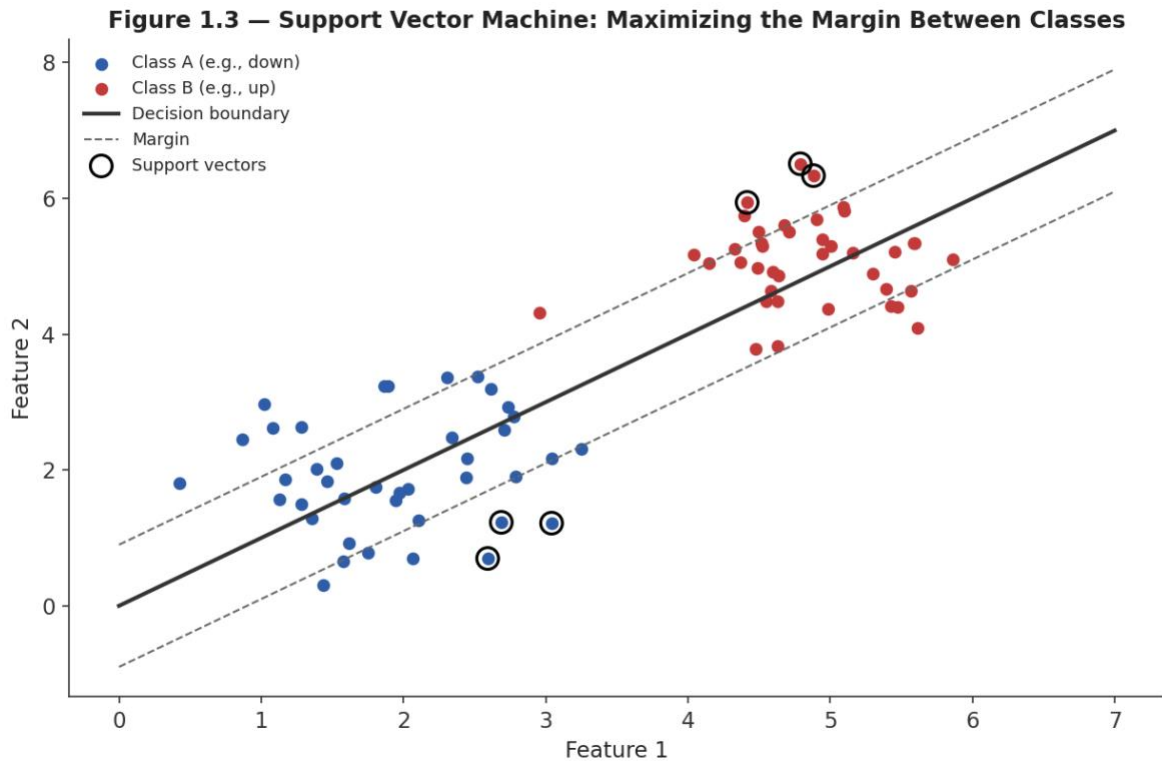


Figure 1.3 — An SVM finds the boundary that maximizes the margin between classes; only the closest points to the boundary (the support vectors, circled) determine its position — every other point could move without changing the fit.

SVMs can be extended, via the **kernel trick**, to find non-linear boundaries by implicitly mapping features into a higher-dimensional space where a linear separator does exist — a mathematically elegant idea, but one that has become less commonly used in quant research in recent years as gradient boosting has generally matched or exceeded SVM performance on tabular financial data while training faster and offering more interpretable outputs; SVMs remain worth knowing for smaller datasets and for problems where the margin-maximization framing is a particularly natural fit.

Part 2 — Unsupervised Learning for Structure Discovery

Unsupervised methods find structure in data without a labelled target — reducing dimensionality, grouping similar observations, or detecting distinct states the market can be in. These are frequently used as a preprocessing step feeding into a supervised model, or as standalone research tools in their own right.

2.1 PCA and Dimensionality Reduction

Principal component analysis (PCA) re-expresses a set of correlated features as a smaller number of uncorrelated **principal components**, each a linear combination of the original features, ordered so that the first component explains the most variance, the second explains the most of what remains, and so on.

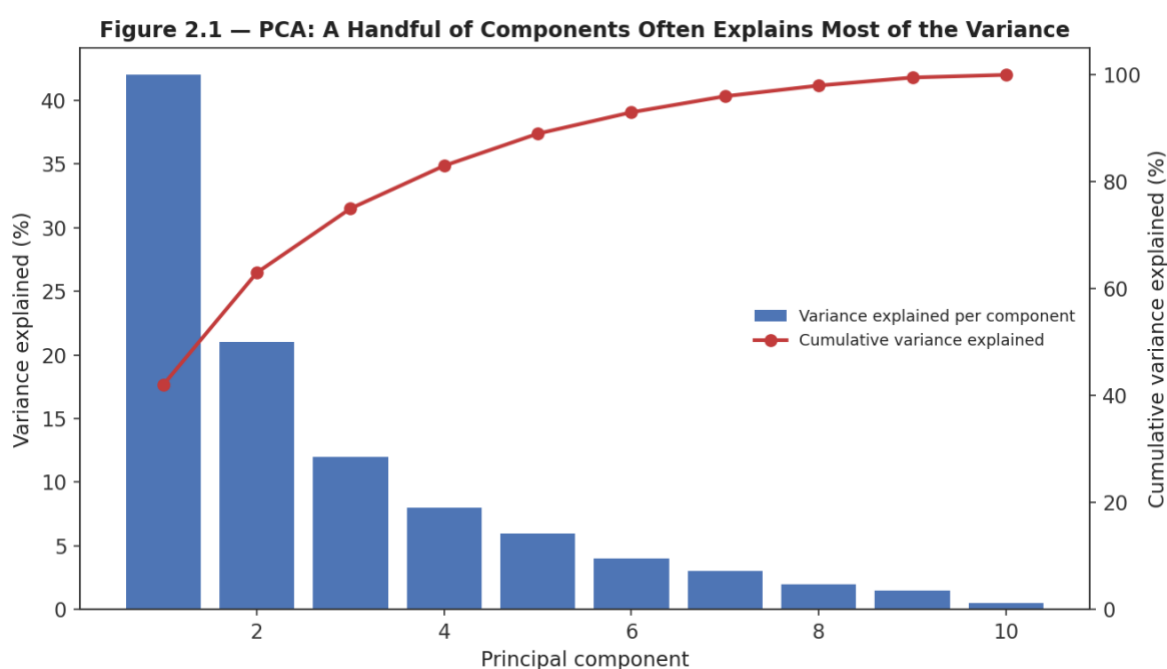


Figure 2.1 — A scree plot shows how much variance each principal component explains; in practice, a handful of components (here, roughly the first four) often capture the great majority of the total variance in a correlated feature set.

In quantitative finance, PCA is used both defensively and constructively: defensively, to reduce a large, highly correlated feature set (many sector, style, and macro exposures, for instance) down to a manageable, less collinear set of inputs before feeding a model; constructively, PCA applied to a universe's returns directly recovers something close to statistical risk factors — the first component of a broad equity universe typically corresponds closely to the overall market factor.

2.2 Clustering

Clustering groups observations — commonly assets, but also time periods or trading days — into clusters based on similarity in feature space, without any predefined group labels. **K-means**, the most common starting point, iteratively assigns each point to its nearest cluster centroid and updates centroids to the mean of their assigned points, until the assignment stabilizes.

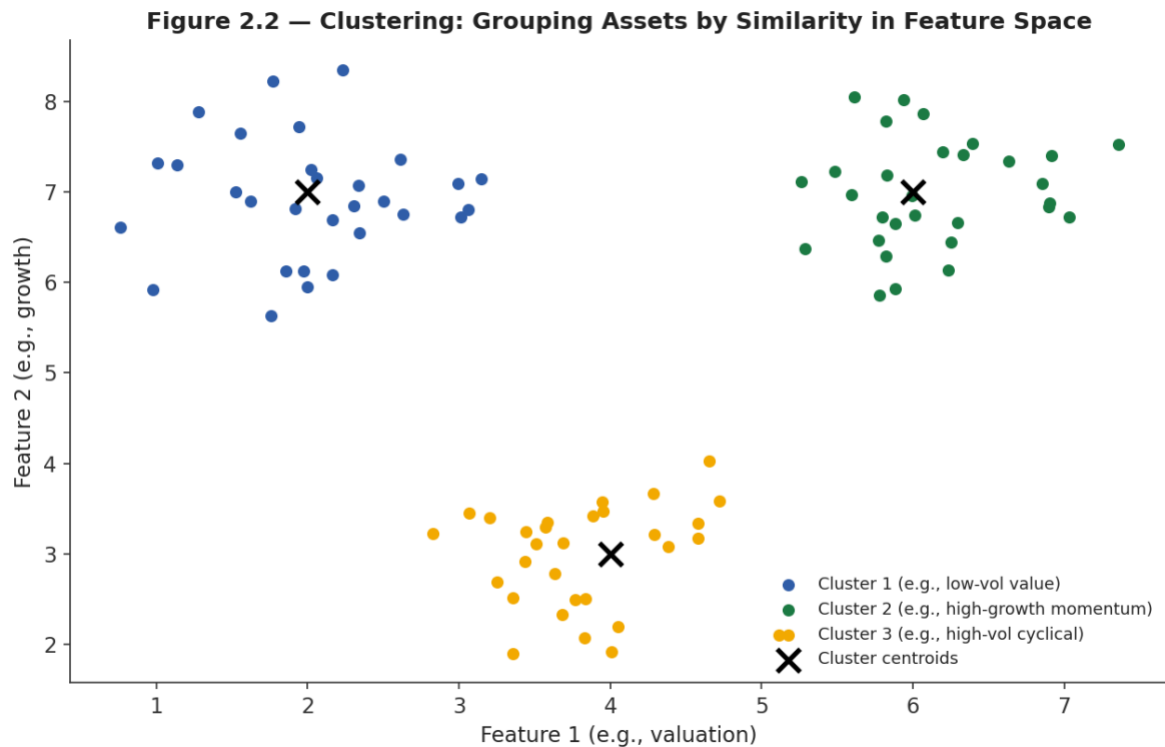


Figure 2.2 — Clustering groups assets by similarity across chosen features (here, valuation and growth); the resulting clusters can reveal groupings that do not correspond neatly to traditional sector or style classifications.

Clustering is used in practice for building peer groups for relative-value analysis that go beyond standard sector classifications, for identifying redundant or highly similar positions within a portfolio for risk purposes, and as a preprocessing step in some regime-identification approaches (Section 2.3), where market days are clustered by their feature characteristics rather than assigned to regimes by a fixed rule.

2.3 Regime Identification

Regime identification applies statistical or ML methods to detect distinct, persistent market states — commonly volatility regimes, but also states defined by correlation structure, liquidity conditions, or macro environment — directly from data, rather than assuming a single, unchanging statistical relationship holds throughout history.

Figure 2.3 — Regime Identification: An Unsupervised Model Detects Volatility Regime Shifts

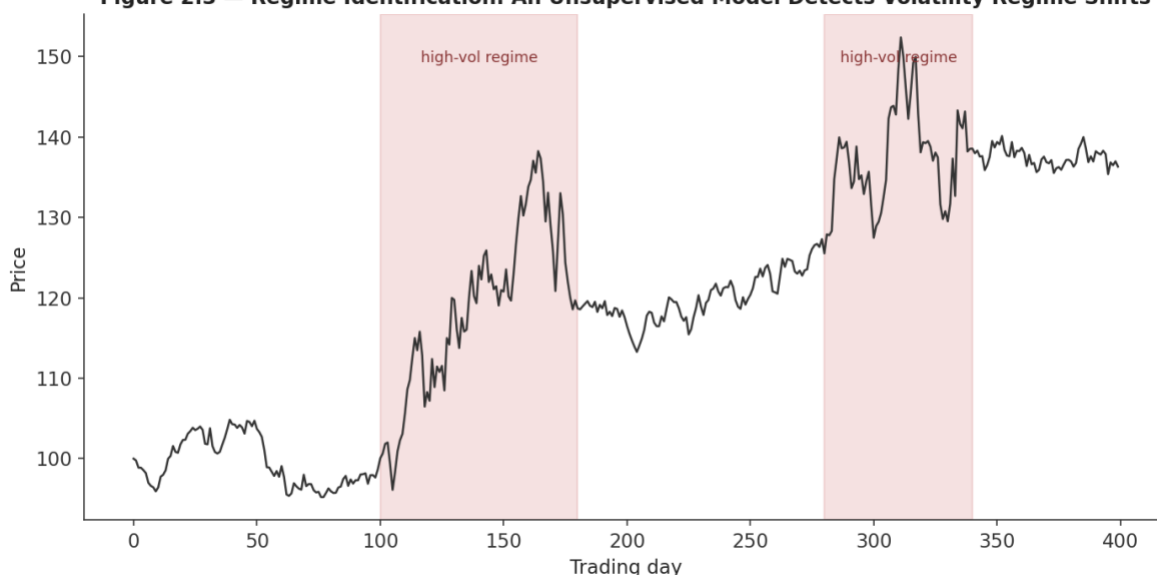


Figure 2.3 — An unsupervised regime model applied to a return series identifies periods of persistently elevated volatility (shaded) without being told in advance where those periods begin and end.

Regime identification connects directly to a theme that runs through this entire module: markets are non-stationary, and a single model trained across a mix of regimes may implicitly learn an average behavior that fits none of the individual regimes well. Explicitly identifying regimes allows either regime-conditional models (a different model, or different parameters, per regime) or regime-aware position sizing (reducing exposure when the current regime is detected to be unfavorable for a given strategy) — both meaningfully more sophisticated than assuming one relationship holds for all time.

Part 3 — Neural Networks (Awareness)

This part is awareness-level throughout: enough to understand what each architecture family is built to capture and roughly when it might be reached for, not enough to design, tune, or deploy one — that is Level 6 territory, and per the prerequisite gate, should only be approached after simpler models have been tried and demonstrably fall short.

3.1 Feed-Forward Networks

A **feed-forward neural network** passes an input through one or more **hidden layers**, each computing a weighted combination of the previous layer's outputs and applying a nonlinear **activation function**, before producing a final output. Stacking layers this way allows the network to represent much more complex, non-linear functions of the input than a single logistic regression could — at the cost of far more parameters to fit, far less interpretability, and correspondingly far greater overfitting risk on the modest, noisy datasets typical of financial prediction problems.

Figure 3.1 — A Feed-Forward Neural Network: Layers of Weighted, Nonlinear Combinations

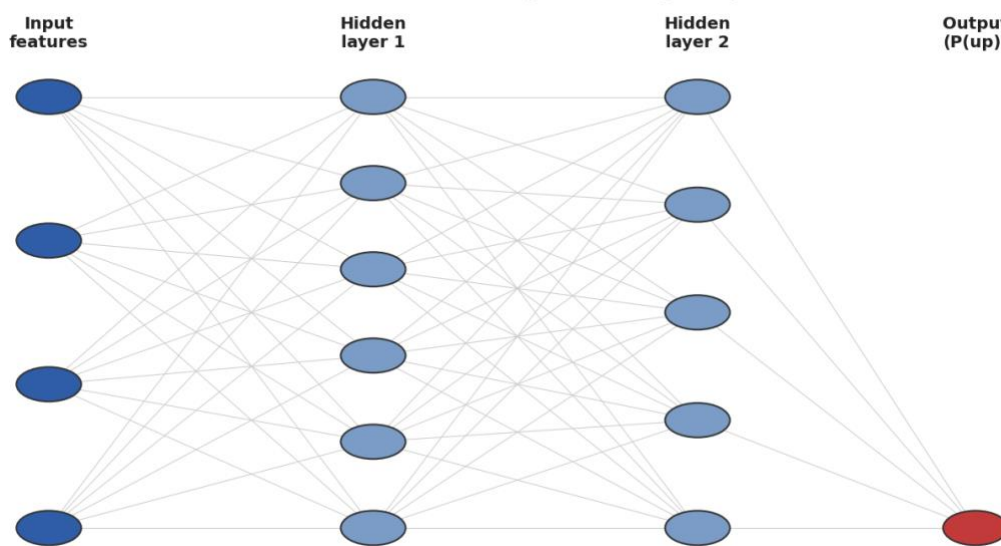


Figure 3.1 — A feed-forward network transforms input features through successive layers of weighted, nonlinear combinations before producing an output — here, a predicted probability of a next-day up move.

For structured, tabular financial features of the kind covered in Part 4, a feed-forward network's practical advantage over gradient boosting is often modest, while its data requirements, tuning burden, and interpretability cost are all higher — which is precisely the situation the prerequisite gate has in mind when it warns against reaching for sophistication before simplicity has been ruled out.

3.2 CNN, RNN/LSTM, and Transformer Architectures (Awareness)

Beyond the plain feed-forward network, three architecture families are built to exploit specific structure in the input data, and are worth recognizing by name and by what each is designed to capture, even without implementation depth at this level.

Figure 3.2 — Neural Network Architecture Families (Awareness-Level Overview)

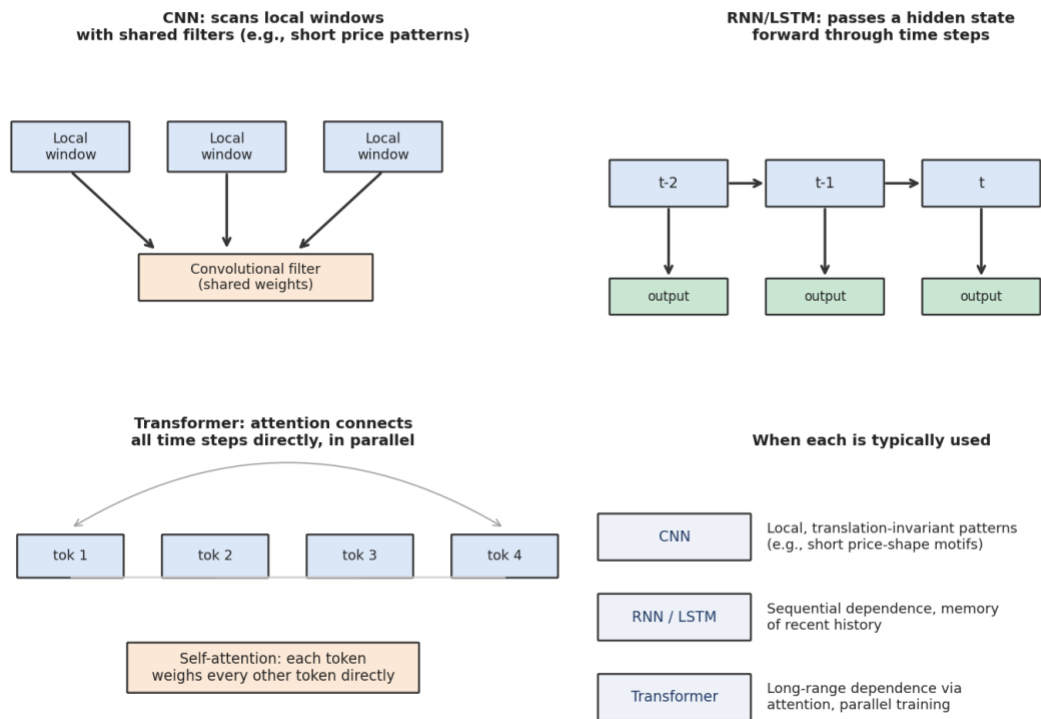


Figure 3.2 — Three architecture families and what each is structurally built to capture: CNNs scan local windows with shared filters; RNNs/LSTMs pass a hidden state sequentially through time; Transformers connect all time steps directly via attention.

A **convolutional neural network (CNN)** scans small, local windows of the input with shared filters, originally developed for images but applicable to short, local patterns in a price or volume series. A **recurrent neural network (RNN)**, and its more capable variant the **LSTM (long short-term memory)** network, processes a sequence step by step, carrying a hidden state forward that in principle summarizes relevant history — well suited to genuinely sequential dependence, at the cost of being slow to train and prone to difficulty learning very long-range dependencies. A **Transformer** instead connects every position in a sequence to every other position directly through a mechanism called **self-attention**, computed in parallel rather than step-by-step — the architecture behind most recent large-scale sequence models, and increasingly explored for long-horizon financial time series, though it brings substantial data and compute requirements that make it a poor first choice for most quant research problems at this level.

Part 4 — The ML Research Pipeline: Features, Labels, and Horizons

This part covers the parts of an ML research pipeline that determine, more than model choice, whether the resulting signal is trustworthy: how raw data becomes features, how a target is defined, and how a model's uncertainty is communicated downstream.

4.1 Feature Engineering

Feature engineering transforms raw data — prices, volumes, fundamentals — into the actual inputs presented to a model: rolling statistics (moving averages, realized volatility), ratios (valuation multiples, volume relative to average), lags (values from n periods ago), and domain-informed transforms specific to the problem (e.g., earnings-surprise magnitude relative to historical surprise volatility for a given company).

Figure 4.1 — The Feature-Engineering Pipeline: From Raw Data to a Feature Matrix

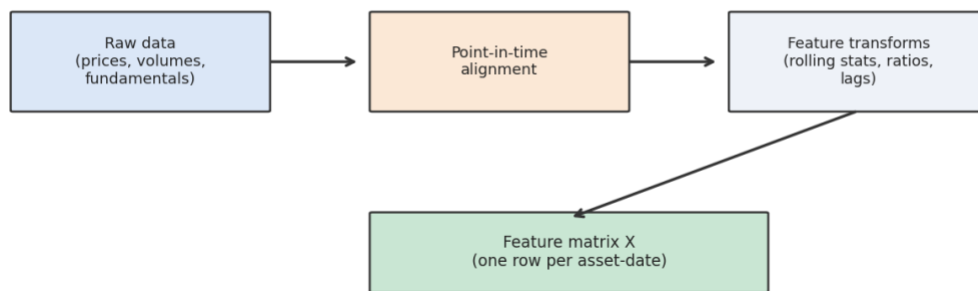


Figure 4.1 — The feature-engineering pipeline: raw data must first be point-in-time aligned (only using information that was actually available as of each date) before any rolling or derived feature is computed.

Common mistake — Using future information in feature construction — for example, an indicator computed with a centred rolling window (using data from both before and after the current date), rather than a trailing window that only looks backward. A centred moving average is smoother and looks better in a static chart, but a model trained on it has silently been given access to future information, exactly the look-ahead bias problem from the Backtesting module, now hidden one layer deeper inside a feature-engineering script rather than in the backtest loop itself.

4.2 Label Construction and Prediction Horizons

Label construction defines the target a supervised model is trained to predict, and is a genuine design decision, not a mechanical afterthought. The simplest choice — the sign of the return over a fixed horizon — is easy to compute but throws away information about magnitude and treats a barely-positive and a strongly-positive return identically.

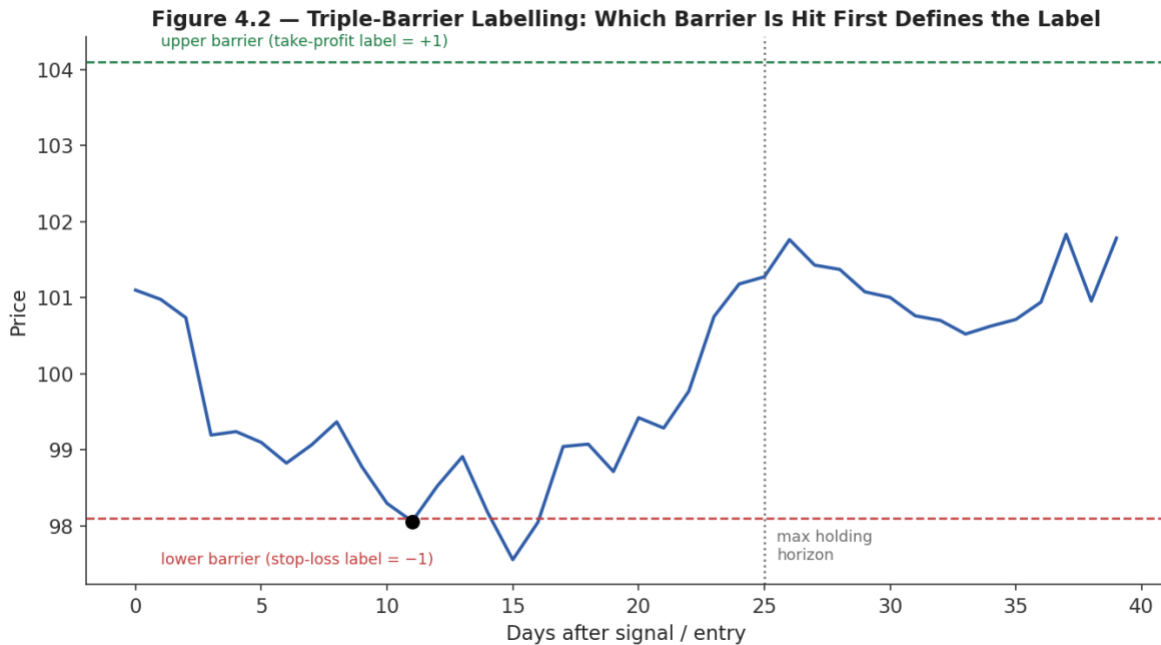


Figure 4.2 — Triple-barrier labelling: a label is assigned based on which of three barriers is hit first — an upper profit-taking barrier, a lower stop-loss barrier, or a maximum holding-period barrier — better reflecting how a position would actually be managed than a fixed-horizon return.

The **triple-barrier method** is a widely used alternative that more realistically reflects how a position would actually be managed: it labels an outcome based on whichever of an upper barrier (take-profit), a lower barrier (stop-loss), or a maximum holding-period barrier is reached first, rather than reading off the return at one fixed, arbitrary horizon regardless of what happened along the way. The choice of **prediction horizon** — how far ahead the label looks — is equally consequential: a short horizon (next-day direction) generates more training examples but a noisier, harder-to-predict target; a long horizon (next-quarter return) generates fewer, more overlapping examples but may capture a more economically meaningful signal, at the cost of the same overlapping-labels complication that motivated purged cross-validation in the Backtesting module.

4.3 Probabilistic Forecasts

A **probabilistic forecast** expresses a prediction as a probability (for classification) or a distribution (for regression) rather than a single point estimate — a model that outputs 'P(up) = 0.62' carries meaningfully more information than one that simply outputs 'up,' because that probability can be used directly to size a position in proportion to model confidence.

Figure 4.3 — Calibration: Does a Predicted 70% Actually Happen 70% of the Time?

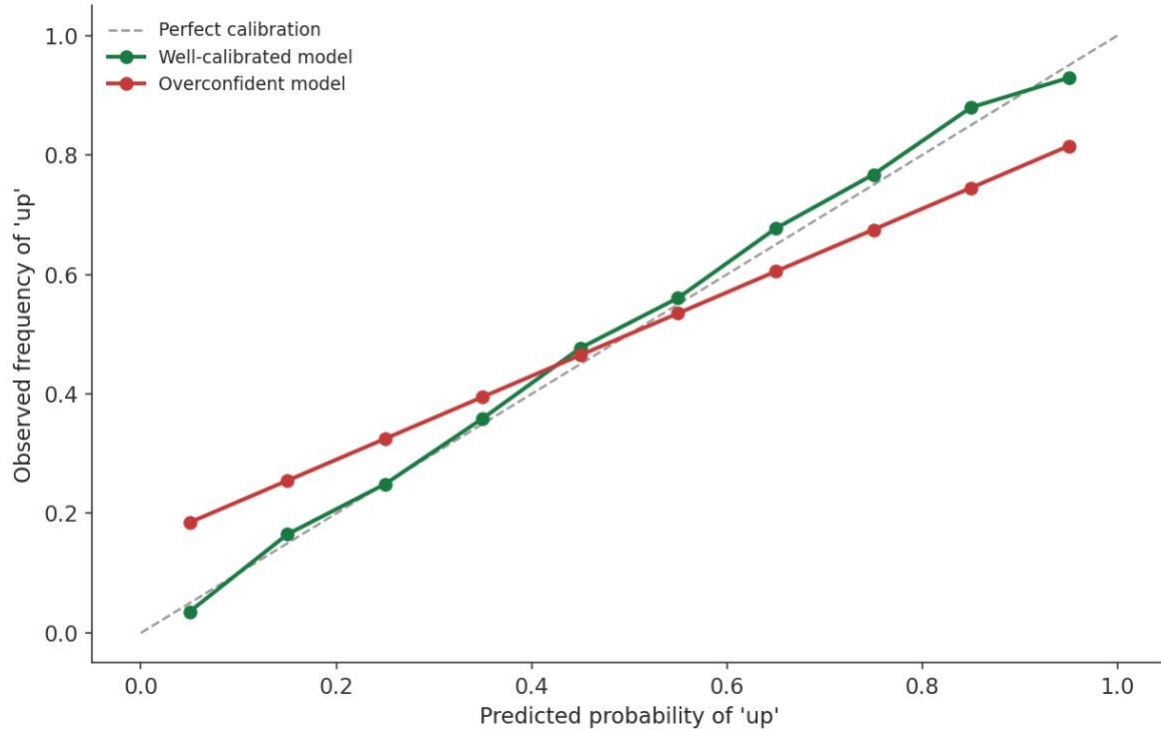


Figure 4.3 — Calibration checks whether a model's stated confidence matches reality: a well-calibrated model's predicted 70% probabilities really do occur about 70% of the time; an overconfident model's stated probabilities are systematically too extreme.

A probabilistic forecast is only useful if it is **calibrated** — a model that claims 90% confidence should be right roughly 90% of the time it makes that claim, not 60% of the time. An uncalibrated model's raw output probabilities cannot be trusted for position sizing even if its directional classification accuracy is genuinely good, because the confidence dial itself is miscalibrated; calibration should be checked explicitly (as in Figure 4.3), and recalibrated if necessary, before a probabilistic output is used for sizing decisions downstream.

Part 5 — Validating and Monitoring ML Models

This closing part covers how an ML signal is validated before deployment and monitored afterward — the section this entire module has been building toward, given the overfitting risk emphasized from the outset.

5.1 Validation for ML Signals

Every validation principle from the Backtesting module applies to ML models without exception, and typically with added force: purged, embargoed walk-forward cross-validation is the default validation scheme for any ML model whose features or labels are built from overlapping time windows — which describes the large majority of genuinely useful financial ML features, given how common rolling-window features and multi-day forward-looking labels both are.

ML research introduces one additional wrinkle beyond a single strategy backtest: **hyperparameter tuning** (choosing tree depth, learning rate, regularization strength, and similar settings) is itself a form of fitting to data, and must be nested inside the validation scheme rather than performed once against the same data used for final evaluation — tuning hyperparameters against the same fold used to report final performance is a direct, if easy to overlook, channel for the same overfitting the rest of this module has been warning about.

```
# Nested purged cross-validation: hyperparameter tuning inside, final eval outside
for outer_train_idx, outer_test_idx in purged_walk_forward_splits(index):
    best_params = None
    best_score = -np.inf
    for params in hyperparameter_grid:
        inner_scores = []
        for inner_train_idx, inner_val_idx in
            purged_walk_forward_splits(outer_train_idx):
            model = fit_model(X[inner_train_idx], y[inner_train_idx], **params)
            inner_scores.append(evaluate(model, X[inner_val_idx], y[inner_val_idx]))
        if np.mean(inner_scores) > best_score:
            best_score, best_params = np.mean(inner_scores), params
    final_model = fit_model(X[outer_train_idx], y[outer_train_idx], **best_params)
    outer_score = evaluate(final_model, X[outer_test_idx], y[outer_test_idx])
```

5.2 Feature Importance and SHAP

Feature importance measures how much each input feature contributes to a model's predictions, both to build trust in the model (do the important features make economic sense?) and to prune uninformative features that add overfitting risk without adding predictive value. Simple importance measures (e.g., how often a feature is used to split in a tree ensemble) are useful but coarse; **SHAP (SHapley Additive exPlanations)** provides a more rigorous, theoretically grounded attribution of each individual prediction to the contribution of each feature.

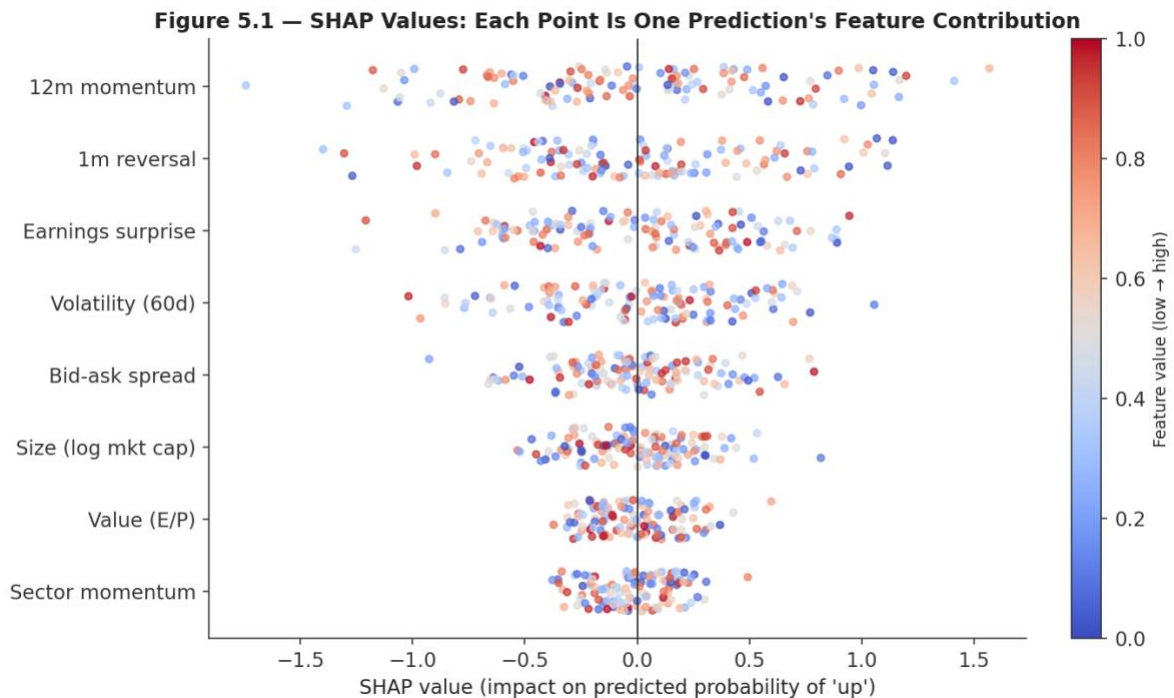


Figure 5.1 — A SHAP summary plot: each dot is one prediction's contribution from one feature, colored by that feature's value; features are ranked by overall impact, and the direction of the color/position relationship reveals whether high or low feature values push predictions up or down.

The practical use of SHAP in this module's practical exercise is a sanity check as much as an interpretability tool: if a gradient-boosted classifier's top SHAP features are '12-month momentum' and 'earnings surprise,' that is consistent with well-understood economic mechanisms and increases confidence the model has found real signal rather than an artifact; if the top feature is something with no plausible economic story, or something suspiciously predictive (a common symptom of leakage), that is a signal to investigate the feature-construction pipeline before trusting the model at all.

5.3 Model Drift and Retraining

Model drift is the degradation of a model's predictive performance over time as the underlying statistical relationship between features and target changes — markets are non-stationary, the relationships a model learned during training are not guaranteed to persist, and an ML model's added flexibility relative to a simple rule-based strategy does not exempt it from this; if anything, a highly flexible model may have fit training-period-specific relationships more tightly, and so may drift faster once those relationships shift.

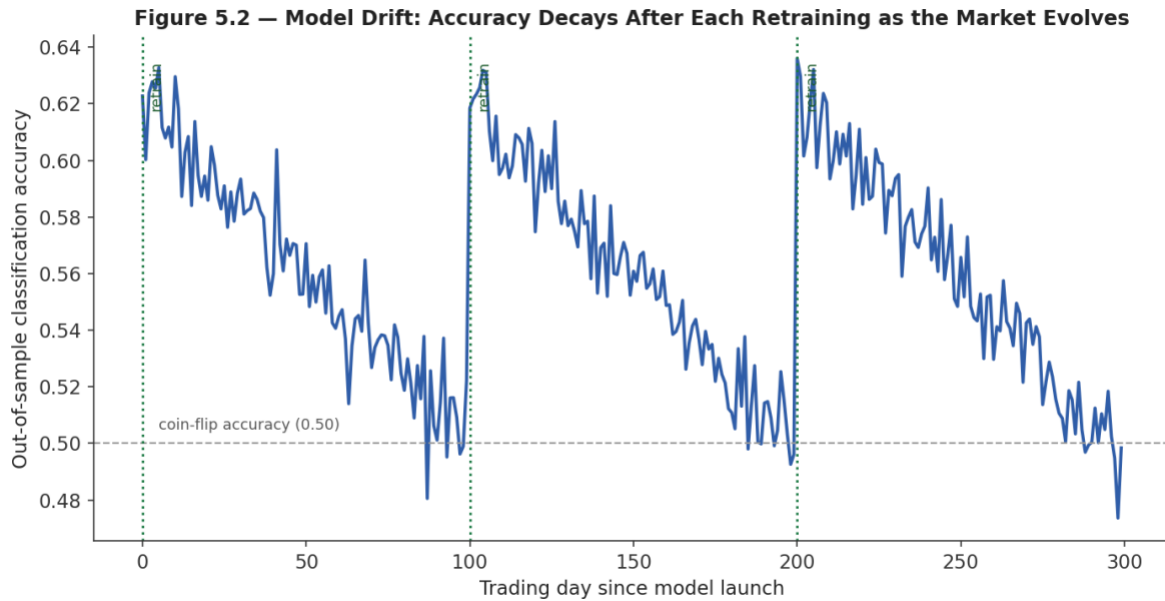


Figure 5.2 — Out-of-sample accuracy decays steadily after each retraining event as market conditions evolve away from the training window, eventually approaching coin-flip performance before the next retrain restores it.

Retraining — refitting the model on more recent data, either on a fixed schedule or triggered by a detected performance decline — is the standard response, but is not a substitute for monitoring: a retraining schedule chosen without reference to how quickly a given model actually drifts is itself an unvalidated assumption, and production ML systems should track live, out-of-sample performance continuously (not just at retraining checkpoints) so that drift is caught as early as possible, consistent with the ongoing model-monitoring responsibility introduced in the Backtesting module's professional-applications section.

Practical Exercise

Build a gradient-boosted classifier to predict next-day direction using 10 engineered features, validate with purged walk-forward cross-validation, and use SHAP to check the top features make economic sense.

Suggested approach

- Construct 10 features from a single, liquid instrument's history — a mix of momentum, reversal, volatility, and volume-based features is a reasonable starting set — ensuring every feature uses only trailing (never centred or forward-looking) windows.
- Define the label as the sign of next-day return, and confirm the label is correctly lagged relative to the features used to predict it.
- Split the data using purged walk-forward cross-validation (Section 5.1), with a purge and embargo length matched to the label's horizon.
- Train a gradient-boosted classifier (e.g., XGBoost) on each fold's training window and evaluate out-of-sample accuracy and a cost-aware backtest, not accuracy alone, on each fold's test window.
- Compute SHAP values on the out-of-sample predictions and inspect whether the top-ranked features have a plausible economic story; if they do not, treat that as a warning sign rather than a footnote.
- Compare the final result against a simple logistic-regression baseline on the same features and the same validation scheme — the gradient-boosted model has not demonstrated value unless it beats this baseline out-of-sample, net of costs.

```
# Practical exercise skeleton
import xgboost as xgb
import shap

results = []
for train_idx, test_idx in purged_walk_forward_splits(index, purge=1, embargo=1):
    model = xgb.XGBClassifier(max_depth=3, n_estimators=200, learning_rate=0.05)
    model.fit(X.iloc[train_idx], y.iloc[train_idx])
    preds = model.predict_proba(X.iloc[test_idx])[:, 1]
    results.append(evaluate(preds, y.iloc[test_idx], prices.iloc[test_idx]))

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X.iloc[test_idx])
shap.summary_plot(shap_values, X.iloc[test_idx]) # inspect: do top features make
sense?
```

Common Beginner Mistakes

- **Using future information in feature construction** — e.g., an indicator computed with a centred rolling window, silently leaking information from after the current date into a feature meant to represent what was knowable as of that date.

- **Treating high in-sample accuracy as evidence of a tradeable edge** without proper out-of-sample and transaction-cost-aware validation — a model can have excellent in-sample accuracy purely from overfitting, and even genuine out-of-sample directional accuracy does not automatically survive real trading costs.
- Reaching for a neural network before establishing that a simpler, interpretable model demonstrably falls short on the same problem.
- Tuning hyperparameters against the same data used for final out-of-sample evaluation, rather than nesting the tuning inside its own validation split.
- Deploying an ML signal into production without a monitoring plan for drift, treating validation as a one-time gate rather than an ongoing discipline.

Advanced Extensions

The following are Level 6 topics, introduced here only by name so you know what to look for when you reach them:

- **Deep learning** — the full treatment of neural network architecture design, training dynamics, and regularization for financial applications where simpler models have been demonstrated to fall short.
- **Reinforcement learning** — learning trading policies directly through reward-based interaction with a (simulated or live) market environment, rather than through supervised prediction of a fixed target.
- **Online learning** — models that update incrementally as new data arrives, rather than being retrained in discrete batches, relevant when drift is fast enough that periodic retraining is insufficient.

Professional Applications

- **Signal generation** — ML models as one input among several into a broader alpha-research process, evaluated against the same validation bar as any other strategy family.
- **Execution-cost prediction** — supervised models trained to predict slippage or market impact for a given order, informing execution-algorithm decisions covered in the Market Microstructure and Backtesting modules.
- **Regime classification** — unsupervised or supervised regime models used to condition position sizing, risk limits, or strategy selection on the currently detected market environment.

Where to Go Next

This module surveyed the standard ML toolkit and, more importantly, the pipeline and validation discipline that determines whether any of it produces a trustworthy result. From here, the curriculum's Risk Metrics module covers how an ML-based signal, once validated, is sized and combined with other strategies, and the Level 6 Advanced Projects revisit deep learning, reinforcement learning, and online learning in full depth, for the specific problems where simpler methods have been demonstrably outgrown.